IMEN574 / IDSC574 · PROGRAMMING FOR AX

14

Deep Learning

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Deep Learning

01 **Learned Features**

Build a nonlinear representation

02 **Differentiable Learning**

Trace the training computation

03 **Architectural Structure**

Match computation to data

01 · Learned Features

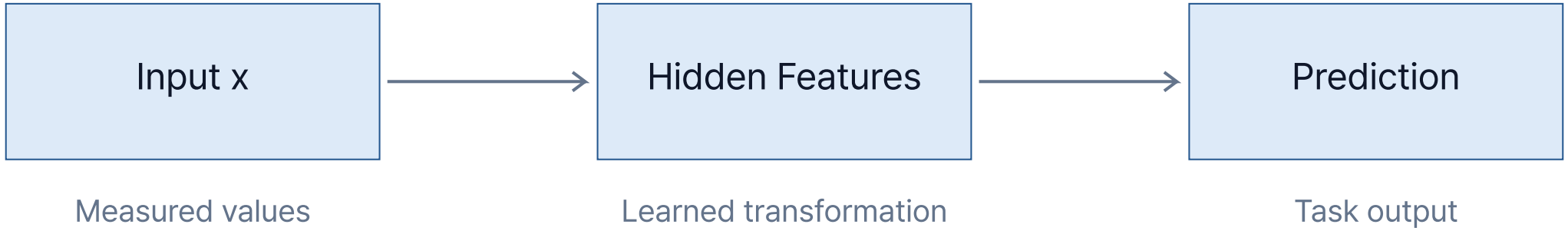
01 **Learned Features**
Build a nonlinear representation

02 **Differentiable Learning**
Trace the training computation

03 **Architectural Structure**
Match computation to data

Learning a Representation

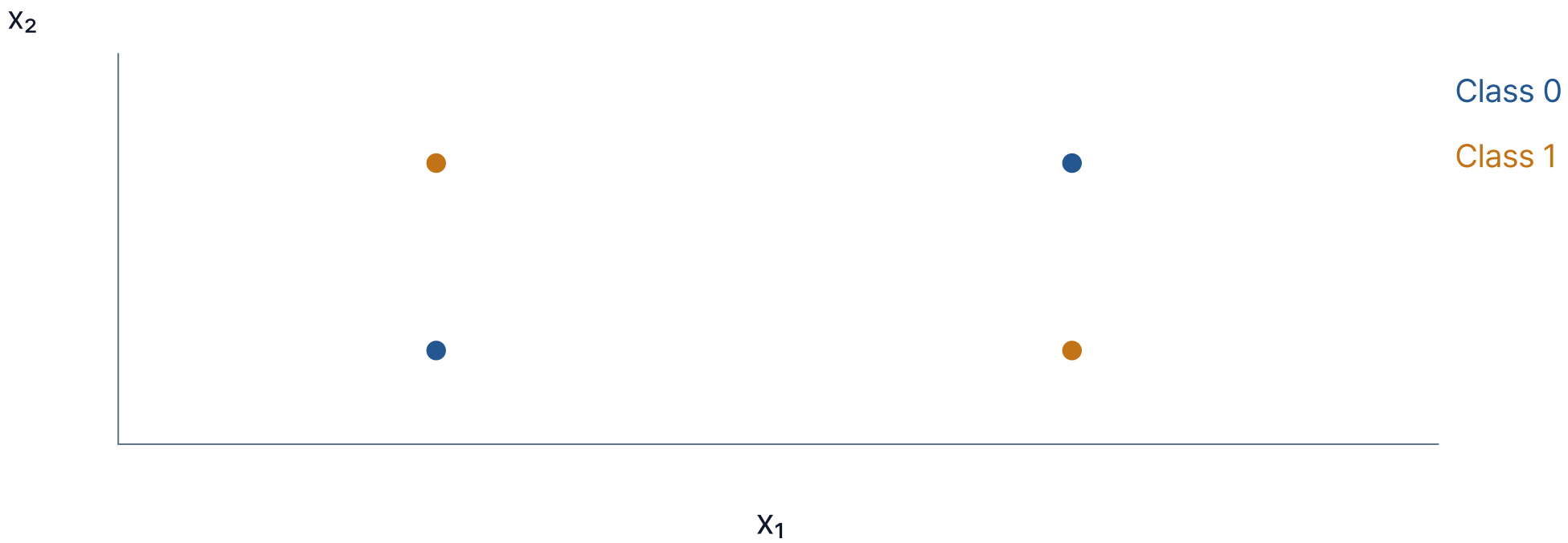
A network learns features together with a predictor



The hidden representation changes during training

A Linear Limitation

One line cannot separate the XOR pattern



Opposite corners share labels that one line cannot separate

Affine Layers

A matrix maps each input into several learned scores

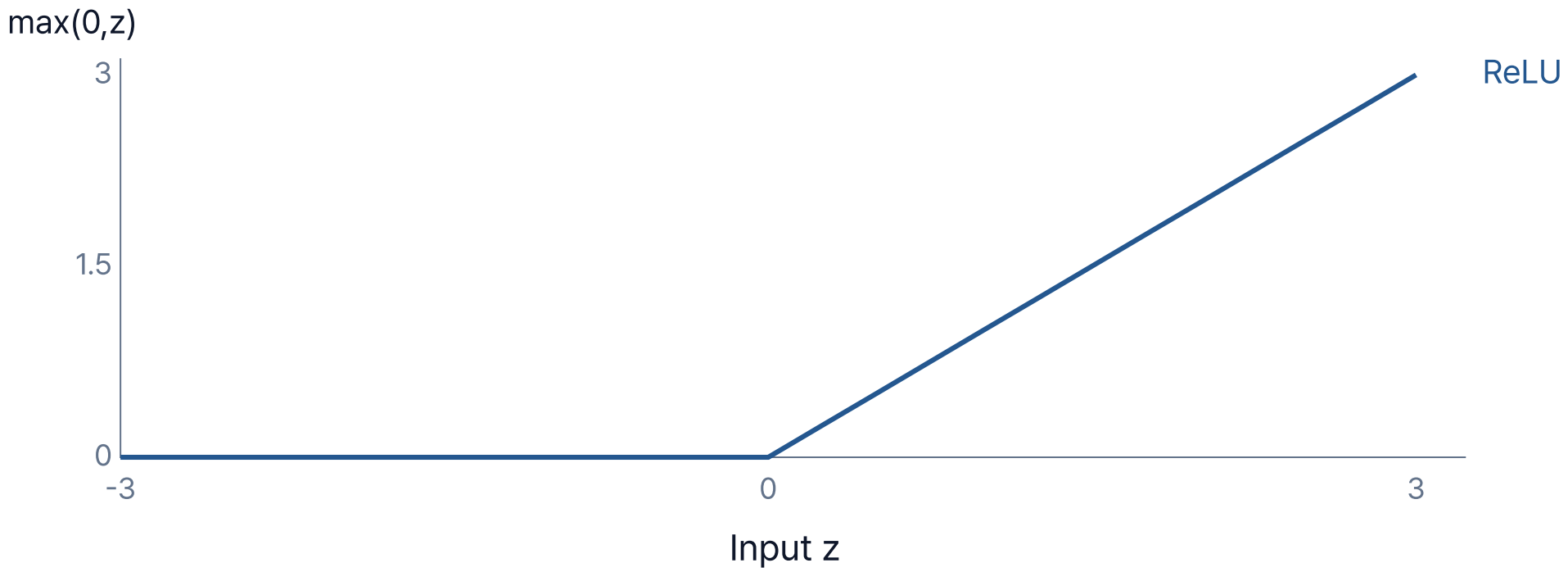
	x_1	x_2
Hidden 1	1	1
Hidden 2	1	1

$$z = Wx + b$$

Each row of W defines one feature calculation

Rectified Linear Units

Keep positive scores and remove negative scores



The kink introduces a nonlinear operation

Nonlinear Feature Construction

Two rectified features can represent the XOR target

	h_1	h_2	$h_1 - 2h_2$
(0,0)	0	0	0
(0,1)	1	0	1
(1,0)	1	0	1
(1,1)	2	1	0

$$h_1 = \mathrm{ReLU}(x_1+x_2),\ h_2 = \mathrm{ReLU}(x_1+x_2-1)$$

The transformed output matches all four XOR labels

Verifying a Tiny Network

Compute every hidden value before training

```
X=np.array([[0.,0.],[0.,1.],[1.,0.],
[1.,1.]])
h1=np.maximum(0,X.sum(1))
h2=np.maximum(0,X.sum(1)-1)
out=h1-2*h2
print(out)
assert np.array_equal(out,[0,1,1,0])
```

```
[0. 1. 1. 0.]
```

Demo: Nonlinear XOR Features

What fails with identity activation? Which row changes when the second bias is zero?

Activation ReLU Second bias -1 Output weight -2

Input	h_1	h_2	Prediction	Target
0,0	0	0	0	0
0,1	1	0	1	1
1,0	1	0	1	1
1,1	2	1	0	0

Activation relu; $h_1 = \text{act}(x_1+x_2)$, $h_2 = \text{act}(x_1+x_2 + -1)$
Prediction = $h_1 + (-2)h_2$ | MSE 0
Four explicit forward passes; no training or model service is called

Four exact forward passes through a two-feature network

Answer: XOR Forward Values

The hidden feature removes the double-positive corner

Input	h_1	h_2	Prediction
0,0	0	0	0
0,1	1	0	1
1,0	1	0	1
1,1	2	1	0

At the default weights, $h_1 - 2h_2$ matches the four labels

Caveat: Representation Is Not Training

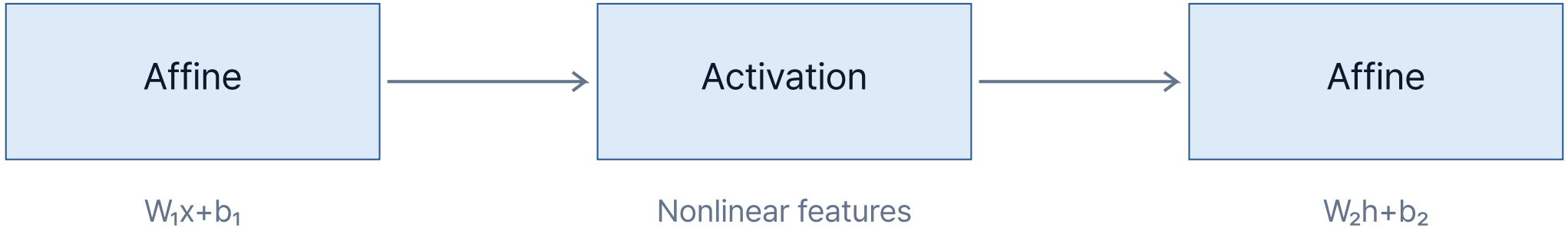
A hand-built solution does not show that an optimizer will find it

Evidence	Established	Still Untested
Exact forward	These weights represent XOR	Learning from initialization
Zero training loss	These four rows fit	Held-out generalization

Correct weights and successful learning are separate claims

Depth and Composition

Compose simple transformations into a richer function



The second layer combines features built by the first

Width and Parameters

Count trainable values before choosing model size

	Input	Output	Weights + Bias
Layer 1	4	8	40
Layer 2	8	3	27

```
Dense layer parameters = d_in × d_out + d_out
```

Two affine layers here contain sixty-seven parameters

Output Layers

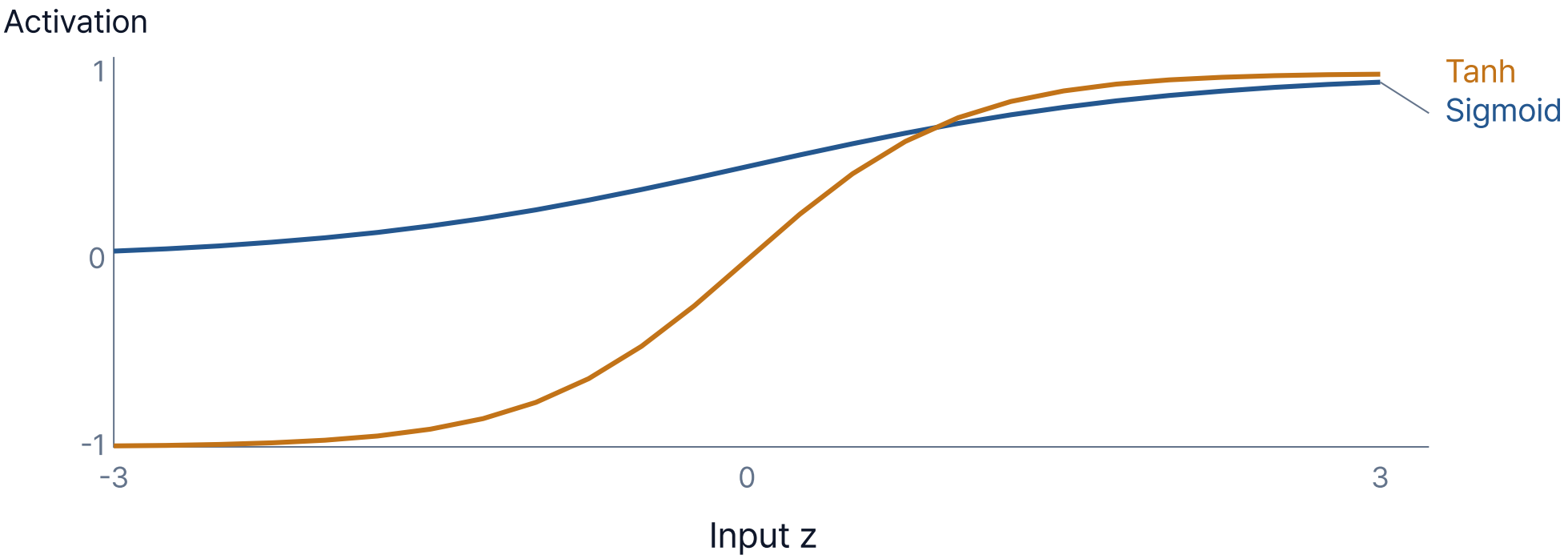
Match the final output to the prediction problem

Task	Output	Loss
Regression	Real values	Squared or absolute loss
Binary	One logit	Binary cross-entropy
Multiclass	C logits	Cross-entropy

The output contract determines the loss interface

Activation Alternatives

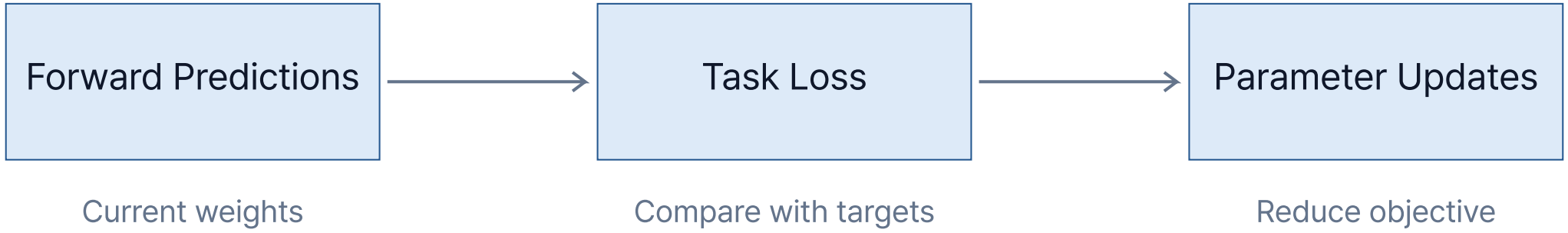
Different nonlinearities change saturation and gradients



Saturating activations flatten for large input magnitudes

Training Objective

Choose weights by minimizing a loss over examples



Representation learning still needs an explicit objective

02 · Differentiable Learning

01 **Learned Features**

Build a nonlinear representation

02 **Differentiable Learning**

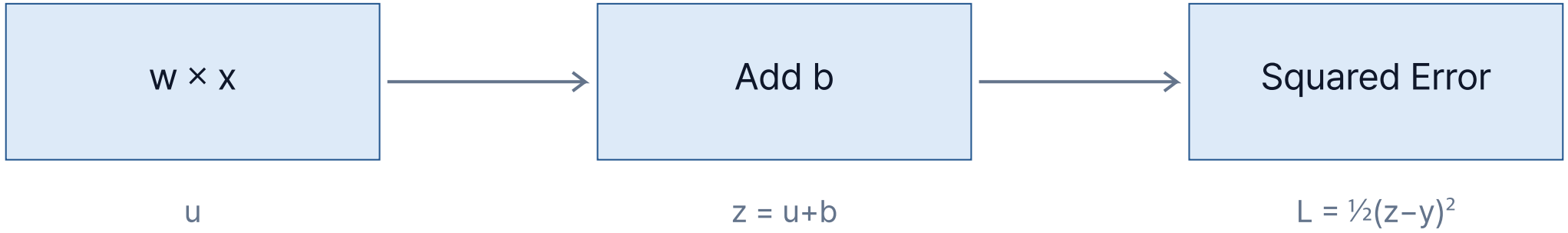
Trace the training computation

03 **Architectural Structure**

Match computation to data

Computation Graphs

Represent intermediate values and dependencies explicitly



The loss depends on w through two intermediate operations

One Forward Pass

Evaluate a scalar model with fixed values

w	x	b	y	z	Loss
1	2	0	3	2	0.5

The current prediction two misses the target three

Local Derivatives

Each operation supplies its own sensitivity

Derivative	Value
L to z	-1
z to w	2
z to b	1

$$\partial L/\partial z = z - y,\ \partial z/\partial w = x$$

The loss falls when this prediction increases

The Chain Rule

Multiply local derivatives along a dependency path

Gradient	Value
∂L/∂w	-2
∂L/∂b	-1

$$\partial L/\partial w = (\partial L/\partial z)(\partial z/\partial w)$$

The weight gradient is minus two for this input

Gradient Accumulation

Shared parameters receive contributions from every use

Path	Gradient Contribution
Example 1	-2
Example 2	1
Sum	-1

Multiple paths add their contributions at a shared parameter

Reverse-Mode Differentiation

Propagate one scalar loss backward through the graph



Start with one

Apply chain rule

Accumulate contributions

One reverse pass computes gradients for many parameters

A Gradient Update

Move parameters against the loss gradient

	Old	Gradient	Step	New
Weight	1	-2	0.1	1.2
Bias	0	-1	0.1	0.1

$$\theta_{new} = \theta - \eta \nabla \theta L$$

The updated parameters raise the prediction toward the target

Finite-Difference Check

Compare a manual gradient with a numerical perturbation

```
def loss(w): return .5*(w*2-3)**2
w=1.; eps=1e-5
numeric=(loss(w+eps)-loss(w-eps))/(2*eps)
analytic=(w*2-3)*2
print(round(numeric,6), analytic)
assert np.isclose(numeric,analytic)
```

```
-2.0 -2.0
```

A Nonlinear Gradient Example

Trace a single ReLU unit before applying an update

Value	x	w	b	v	c
Initial	2	1	−1	2	0.5
Forward	z=1	h=1	ŷ=2.5	target=3	L=0.125

The active hidden unit passes a nonzero gradient

Demo: A Nonlinear Backpropagation Loop

Which gradient stops when the hidden unit is inactive? Does a larger step lower loss?

Input x 2 Step η 0.05 Updates 1

x		z = wx+b		h = ReLU(z)		ŷ = vh+c		L = ½(ŷ−3)²
2	→	1.250	→	1.250	→	3.056	→	0.001582

w=1.100, b=-0.9500, v=2.025, c=0.5250

Explicit one-hidden-unit ReLU model | updates 1; loss 0.1250 → 0.001582
Last gradient: dw -2, db -1, dv -0.5000, dc -0.5000
Target = 3; ReLU derivative is defined as zero at z ≤ 0 in this teaching implementation

Explicit forward, chain-rule gradients, and parameter updates

Answer: One Nonlinear Update

The same gradients change all four trainable values

State	w	b	v	c
Before	1	−1	2	0.5
Gradient	−2	−1	−0.5	−0.5
After η=0.05	1.1	−0.95	2.025	0.525

The new forward prediction is 3.05625 and loss is 0.001582

Caveat: An Inactive ReLU

An inactive hidden unit blocks its upstream gradient

At x=0	Hidden Value	dw / db / dv	dc
Initial state	0	0 / 0 / 0	−2.5

The output bias can move while the hidden path stays inactive

03 · Architectural Structure

01 Learned Features

Build a nonlinear representation

02 Differentiable Learning

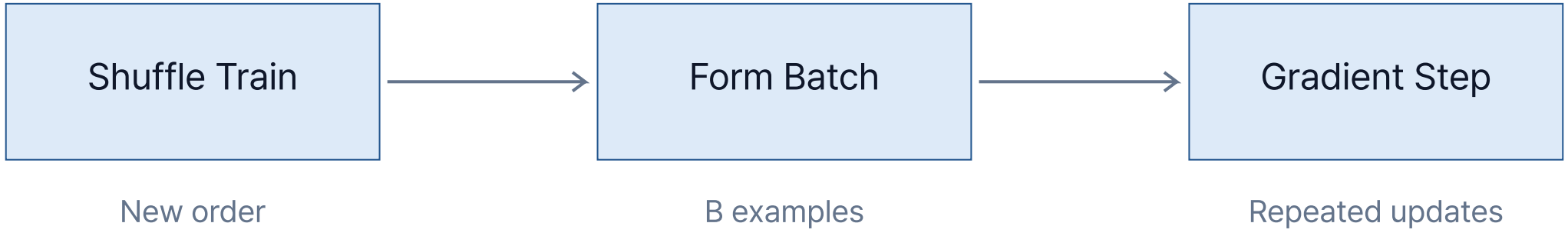
Trace the training computation

03 **Architectural Structure**

Match computation to data

Mini-Batch Learning

Estimate a loss gradient from a subset of examples



Each step uses only the selected batch

Gradient Noise

Small batches produce different update estimates

	g_1	g_2	Mean g
Batch A	-2	1	-0.5
Batch B	-1.8	-0.2	-1.0

Different batches can point in different local directions

Vanishing Gradients

Repeated small derivatives shrink a backward signal

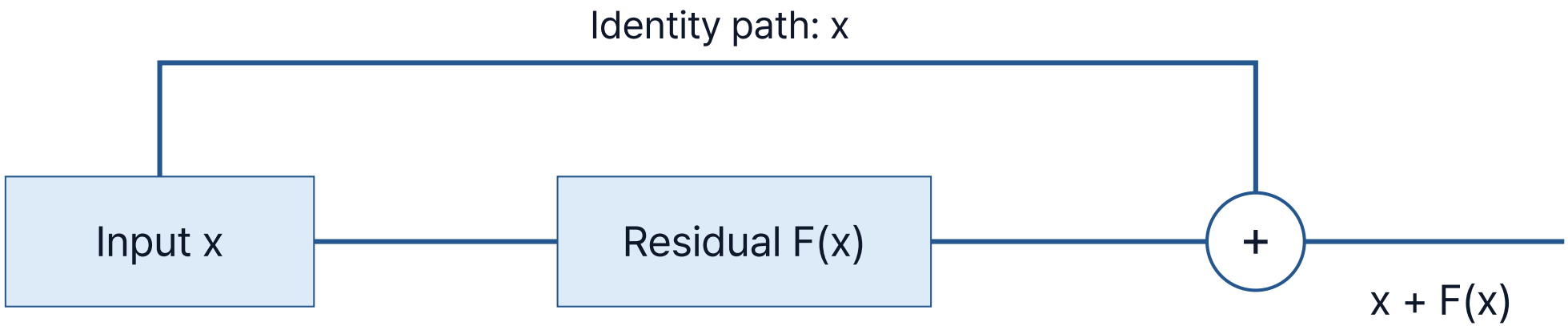


Product of 0.25 derivatives

The product shrinks rapidly across repeated derivatives

Residual Connections

An identity path gives another route for the signal



$$dy/dx = 1 + dF(x)/dx$$

The derivative includes an identity contribution

Convolutional Structure

Share a local pattern detector across positions

Input	1	2	3	4
Kernel	$[-1, +1]$ · stride 1 · no padding			
Output		1	1	1

Valid correlation returns three local differences

Attention as an Architectural Choice

Combine representations using content-dependent weights

	Value A	Value B	Value C
Query A	0.7	0.2	0.1
Query B	0.1	0.6	0.3
Query C	0.2	0.2	0.6

Different queries form different weighted combinations

Architecture and Inductive Bias

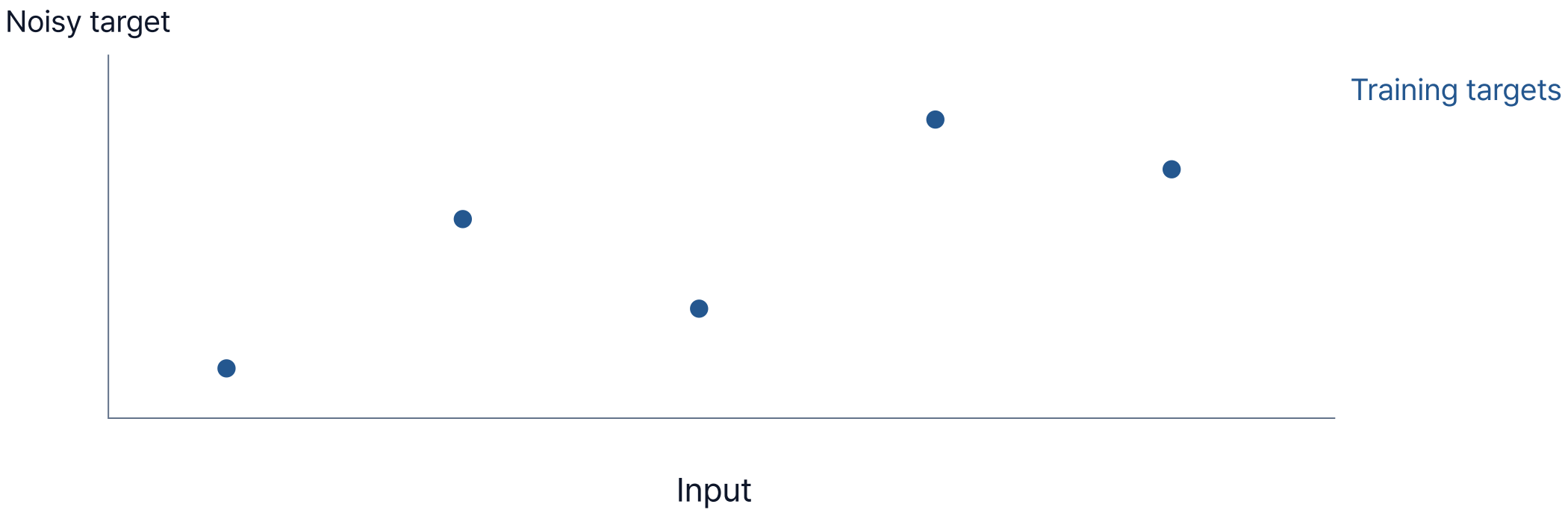
Structure restricts how information is combined

Architecture	Shared Assumption
MLP	Dense feature combinations
CNN	Local patterns repeated in space
Transformer	Content-dependent interactions

An architecture encodes a useful structural preference

Caveat: Capacity and Evidence

More parameters can fit noise as well as signal



A finite noisy sample does not reveal one unique function

Scaling Resources

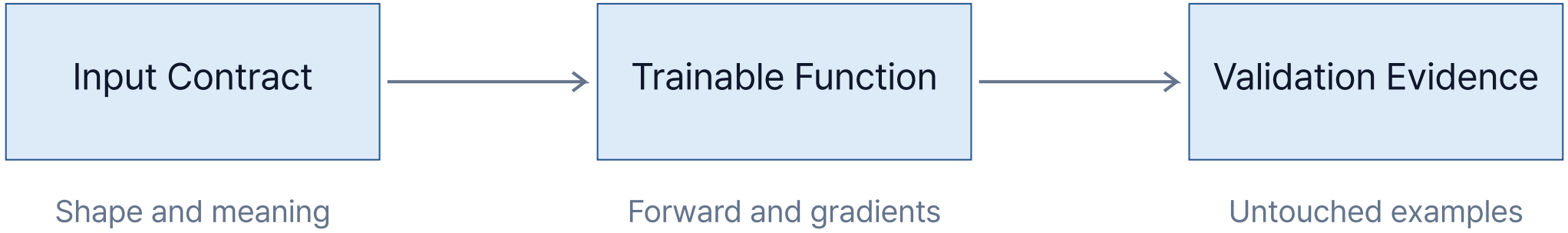
Data, parameters, and compute constrain one training run

Budget	Constraint
Data	Examples and coverage
Parameters	Representational capacity
Compute	Steps and precision

Increasing only one resource can leave another bottleneck

A Complete Learning Contract

Connect the input shape, objective, and validation target



Deep learning remains an explicit, testable computation

Takeaway: Deep Learning

01 Hidden transformations learn task-relevant features

02 Backpropagation applies the chain rule

03 Architecture and validation constrain useful learning